Project Documentation

FitBuddy – AI Fitness Plan Generator

1. Introduction

Project documentation is an important part of the software development process. It provides a detailed explanation of the system architecture, development process, and instructions for installing and running the application. Proper documentation helps developers, users, and administrators understand how the system works and how it can be maintained or extended in the future.

The FitBuddy project documentation explains the structure of the application, the technologies used, installation procedures, and the overall workflow of the system.

This documentation serves as a reference guide for developers who wish to understand, modify, or deploy the application.

---

2. System Overview

FitBuddy is an AI-powered fitness planning application that generates personalized workout plans based on user input. The system collects basic fitness information from users and uses artificial intelligence to generate structured workout routines and nutrition tips.

The system includes the following major components:

- User Interface
- Backend API
- AI Model Integration
- Database System

Each component plays a specific role in delivering the final functionality of the application.

---

3. System Architecture

The FitBuddy system follows a layered architecture where each component interacts with others through defined interfaces.

Architecture Overview

```
User Interface
      ↓
FastAPI Backend
      ↓
AI Model
      ↓
Database
```

User Interface

The frontend is responsible for collecting user inputs and displaying the generated workout plans. It is built using HTML and CSS along with Jinja2 templates for dynamic content rendering.

Backend API

The backend is developed using FastAPI. It handles user requests, processes input data, communicates with the AI model, and interacts with the database.

FastAPI was chosen because of its high performance and ease of integration with Python-based AI libraries.

AI Model Integration

Artificial intelligence is used to generate personalized workout plans and nutrition recommendations.

The backend sends structured prompts to the AI model based on the user's input. The model processes the prompt and returns a workout plan tailored to the user's fitness goals.

Fallback responses are implemented to ensure the system continues to function even if the AI API fails.

Database

The database stores user information and generated workout plans.

Typical stored data includes:

- User name
- Age
- Weight

- Fitness goal
- Workout intensity
- Generated workout plans
- Updated workout plans

The database ensures that all user data is preserved and accessible for administrative purposes.

---

4. Project Directory Structure

The FitBuddy project follows a modular directory structure to keep the code organized and maintainable.

Example project structure:

```
FitBuddy
│
├── app
│   ├── main.py
│   ├── routes.py
│   ├── database.py
│   ├── gemini_generator.py
│   ├── gemini_flash_generator.py
│   ├── updated_plan.py
│
│   ├── templates
│   │   ├── index.html
│   │   ├── result.html
│   │   └── all_users.html
│
│   └── static
│       └── style.css
│
├── requirements.txt
├── .env
└── README.md
```

This structure separates backend logic, templates, and static resources.

---

5. Installation Guide

The following steps explain how to install and run the FitBuddy application locally.

Step 1: Clone the Repository

```
git clone https://github.com/username/fitbuddy-ai
```

Step 2: Navigate to Project Folder

```
cd fitbuddy-ai
```

Step 3: Install Required Libraries

```
pip install -r requirements.txt
```

Step 4: Configure Environment Variables

Create a `.env` file and add the required API keys and database connection strings.

Example:

```
GEMINI_API_KEY=your_api_key
MONGO_URL=your_database_connection
```

Step 5: Run the Application

Start the FastAPI server using the following command:

```
uvicorn app.main:app --reload
```

Step 6: Open the Application

Open a web browser and visit:

```
http://127.0.0.1:8000
```

The FitBuddy homepage will appear where users can generate workout plans.

---

6. Deployment Guide

The application can be deployed on cloud platforms such as Render or Railway.

Deployment steps include:

1. Upload the project to GitHub
2. Connect the repository to the deployment platform
3. Configure environment variables
4. Deploy the application

Once deployed, users can access the application through a public URL.

7. Maintenance and Future Updates

The modular design of the FitBuddy system allows easy maintenance and future improvements.

Possible future enhancements include:

- User authentication system
- Fitness progress tracking
- Mobile application support
- AI chatbot for fitness advice
- Integration with wearable fitness devices

These features could further enhance the usefulness of the system.

---

8. Summary

The documentation phase ensures that the FitBuddy project is well organized and understandable for future developers and users. By providing clear explanations of the system architecture, installation steps, and deployment process, the documentation makes it easier to maintain and extend the application.

Well-structured documentation is essential for the long-term success and scalability of any software project.